Airline Route Network Analysis
Technical Walkthrough

A step-by-step explanation of how the project was built, written for my own reference so I can walk through and explain every decision in an interview.

Srushti Gandhi

Pipeline Overview

The project runs as six sequential scripts, each reading the previous step's output. This mirrors a real analytics pipeline: raw data in, clean/scored data out, then visualization on top.

Script	What it does	Input → Output
`01_generate_data.py`	Builds the route dataset	— → raw/t100_style_route_data.csv
`02_data_pipeline.py`	Cleans data, engineers metrics	raw CSV → processed/routes_clean.csv, routes_annual.csv
`03_network_analysis.py`	Graph/network analysis	routes_annual.csv → airport_centrality.csv, network_summary.json
`04_profitability_model.py`	Scores + classifies routes	annual + centrality → routes_scored.csv
`05_visualizations.py`	Builds interactive dashboard	scored data → dashboard.html + interactive charts
`06_static_charts_for_pdf.py`	Builds PNGs for this PDF	scored data → static PNG charts

Step 1 — Generate the Dataset

Script: `01_generate_data.py`

I built a synthetic dataset shaped like the real BTS T-100 Domestic Segment dataset, since I didn't have live internet access to pull the real file in this environment. The structure: 5 hub airports + 20 spoke airports, connected in a hub-and-spoke pattern (every hub flies to every spoke, hubs connect to each other, plus a few high-demand spoke-to-spoke leisure routes), across 4 quarters.

For each route-quarter, I simulated: departures, seats, load factor (with seasonality built in), passengers, average fare (using a yield-per-mile model where shorter/thinner routes charge more per mile), revenue, and operating cost (using a cost-per-available-seat-mile model where costs go down on longer routes — the real 'stage-length effect' in airline economics, where fixed costs like taxi/boarding amortize over more miles).

Why it matters: I made sure the simulation had real airline economics baked in (stage-length cost effect, seasonal leisure vs. business demand, hub premium pricing) rather than just random numbers — that's what makes the downstream analysis meaningful instead of noise. On a real job, this step would just be pd.read_csv() on the actual BTS download.

Step 2 — Clean the Data & Engineer Metrics

Script: `02_data_pipeline.py`

Cleaning:

- Dropped rows with nulls or non-physical values (zero/negative passengers, seats, or distance)
- Dropped any row with a load factor over 100% (a data-quality guardrail, even though the simulator shouldn't produce this)

Metrics engineered (the same ones a real network-planning analyst uses):

- **ASM** (Available Seat Miles) = seats × distance — total flying capacity
- **RPM** (Revenue Passenger Miles) = passengers × distance — capacity actually sold
- **RASM** = revenue ÷ ASM — revenue efficiency per unit of capacity flown
- **Yield** = revenue ÷ RPM — price per passenger-mile actually flown
- **Operating margin** = (revenue – op. cost) ÷ revenue

Then I rolled the 4 quarterly rows per route up into one annual row per route (groupby origin/dest, sum the volume fields, recompute the ratio fields from the summed totals rather than averaging the ratios directly — averaging percentages would have been a mistake here). I also computed a seasonality index per route as (max quarter – min quarter) ÷ mean, to flag which routes swing the most seasonally.

Why it matters: This is the step I'd get asked about most in an interview — why RASM/yield instead of just revenue, and why I recomputed ratios post-aggregation instead of averaging them. Recomputing avoids Simpson's-paradox-style distortion where a route with one huge quarter and three tiny ones gets an average margin that doesn't reflect its actual annual economics.

Step 3 — Network / Graph Analysis

Script: `03_network_analysis.py` (uses `networkx`)

This is the differentiating piece of the project — most portfolio analytics projects never touch graph theory. I modeled the airline's route map as a directed, weighted graph: airports are nodes, routes are edges, and each edge is weighted by passenger volume.

Three analyses on top of the graph:

1. **Centrality** — degree centrality (how many routes an airport has, weighted by volume), betweenness centrality (how often an airport sits on the shortest path between other airport pairs), and eigenvector centrality (an airport's influence, weighted by how influential its neighbors are — this is the same family of algorithm behind Google's original PageRank). Eigenvector centrality is what I used as the network value score, since it captures 'strategically important,' not just 'busy.'
2. **Resilience simulation** — for each hub, I removed it from the graph and measured what % of the remaining airports could still reach each other. This is a simple stand-in for a real disruption-planning question: if weather shuts down a hub, how much of the network still functions?
3. **New-route candidate detection** — I looked at every spoke-to-spoke pair with no existing direct route, and estimated the 'implied connecting demand' by looking at how many passengers already fly through a shared hub to get between those two cities (assuming roughly 35% of hub-connecting passengers would switch to a nonstop if one existed). Pairs with high implied demand and no current nonstop are the new-route candidates.

Why it matters: This is the section I'd walk an interviewer through in the most detail, since it's the part that shows I can think beyond flat tables. A revenue leaderboard alone would never have surfaced Seattle as more structurally important than Atlanta, or flagged LA–Phoenix as a missing route — both only show up once you model the network as a graph.

Step 4 — Profitability / Route Health Scoring Model

Script: `04_profitability_model.py`

I combined four signals into one composite **Route Health Score** (0–100), each min-max normalized across the portfolio first so they're comparable:

Signal	Weight	Why this weight
Operating margin	40%	Profitability is still the primary driver of a real add/cut decision
Load factor	25%	Operational efficiency — a route can be profitable but fragile if it's barely full
Q1→Q4 passenger growth	20%	Momentum matters — a growing route is worth more than a flat one at the same margin
Network value (centrality)	15%	Strategic fit — some routes are worth protecting even at a thinner margin, because they feed the

I classified routes into EXPAND / MAINTAIN / MONITOR / CUT by quantile within the portfolio (top 20% / next 30% / next 30% / bottom 20%) rather than a fixed score cutoff — this matters because a fixed cutoff (e.g. 'score ≥ 70 = EXPAND') can produce an empty or lopsided bucket depending on how the portfolio happens to be distributed. Quantile-based classification is also how real network-planning teams actually

screen a portfolio each quarter: relative to the rest of the current network, not against an arbitrary absolute bar.

Step 5 — Dashboard & Visualizations

Scripts: `05_visualizations.py` (interactive, Plotly) and `06_static_charts_for_pdf.py` (static, matplotlib)

I built two versions of the same charts on purpose: interactive Plotly charts combined into a single-page HTML dashboard (for the live portfolio-site link, and as a stand-in for what would be a Power BI dashboard in a real company environment with Power BI access), and static matplotlib PNGs for this PDF and for the project card image on my portfolio site.

The dashboard includes:

- A US route map, color-coded by recommendation, line thickness scaled to profit
- Top/bottom routes by profit and margin
- Seasonal demand trend
- Airport centrality ranking
- Recommendation mix (donut)
- New route candidates

Why it matters: I also exported a Power BI-ready CSV (outputs/exports/route_action_list.csv) alongside the HTML dashboard, so this project can show up two ways on my resume/portfolio: as a live clickable dashboard link, and as a 'built the model in Python, visualized in Power BI' story if I want to actually open the export in Power BI Desktop and screenshot it.

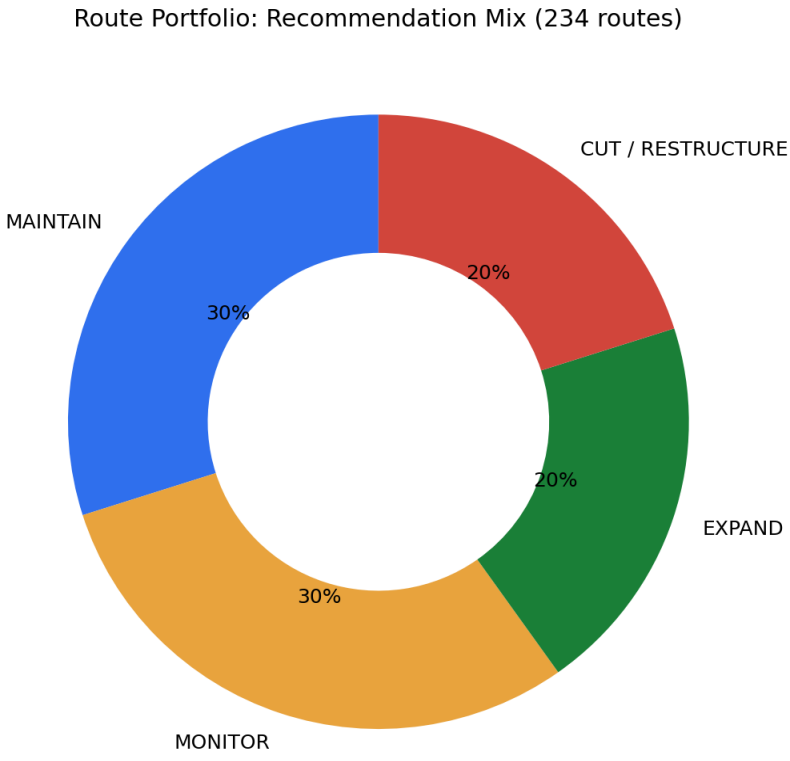

Figure — Route portfolio recommendation mix, as shown on the dashboard

How to Re-Run or Extend This

To rebuild everything from scratch:

```
pip install pandas numpy networkx plotly matplotlib
python scripts/01_generate_data.py
python scripts/02_data_pipeline.py
python scripts/03_network_analysis.py
python scripts/04_profitability_model.py
python scripts/05_visualizations.py
python scripts/06_static_charts_for_pdf.py
```

To use real data instead of the synthetic set: download T-100 Domestic Segment data from transtats.bts.gov, save it in the same column shape as data/raw/t100_style_route_data.csv, and re-run from Step 2 onward.

Possible extensions if I want to go further with this later:

- Add a real ML model (e.g. predict next quarter's load factor per route) instead of the rule-based scoring model
- Pull live BTS data and compare synthetic vs. real network structure
- Add fuel price sensitivity to the cost model
- Build the actual .pbix Power BI file once I have access to Power BI Desktop